

Pair-In, Pair-Out: Latent Multi-Token Prediction for Efficient LLMs

Wenhui Tan¹, Minghao Li², Xiaoqian Ma², Siqi Fan³,
Xiusheng Huang⁴, Liuji Zhang², Ruihua Song¹, Weihang Chen²

¹Gaoling School of Artificial Intelligence, Renmin University of China,

²AI Platform, Xiaohongshu Inc.,

³University of Electronic Science and Technology of China,

⁴Institute of Automation, Chinese Academy of Sciences

Correspondence: Ruihua Song and Weihang Chen

Project Page: [GitHub.com/RedAI-Infra/PIPO](https://github.com/RedAI-Infra/PIPO)

Abstract

Long chain-of-thought reasoning has made autoregressive decoding the dominant inference cost of modern large language models. Existing methods target either the input side (latent compression) or the output side (speculative decoding and multi-token prediction, MTP), but the two lines of work have been pursued independently. Moreover, output-side methods must incur an expensive verifier pass to validate the unreliable draft tokens predicted by MTP. To address these issues, we propose **Pair-In, Pair-Out (PIPO)**, which unifies both sides by viewing a latent compressor and an MTP head as mirror-image operations: the compressor folds two input tokens into one latent representation, while the MTP head unfolds one hidden state into one additional output token. To remove the verifier cost without sacrificing reliability, PIPO trains a lightweight confidence head that decides whether draft tokens should be accepted. We observe that On-Policy Distillation (OPD) naturally matches the rejection-sampling criterion of speculative decoding, so the confidence head can be trained alongside OPD with negligible extra cost. Experiments on AIME 2025, GPQA-Diamond, LiveCodeBench v6, and LongBench v2 with Qwen3.5-4B and 9B backbones show that PIPO improves pass@4 over regular decoding by up to +7.15 points, while delivering up to $2.64\times$ first-token-latency and $2.07\times$ per-token-latency speedups.

1 Introduction

Large language models (LLMs) are increasingly used as reasoners on complex tasks such as mathematics and coding, typically by generating long chains of intermediate reasoning tokens before producing a final answer (Wei et al., 2022; Jaech et al., 2024; Guo et al., 2025). While this paradigm improves accuracy, it makes inference expensive. Under standard autoregressive decoding, an LLM

emits one token per forward step, and each step depends on all previously generated tokens. Long reasoning traces therefore translate directly into long decoding latency, and the per-token cost has become the dominant inference bottleneck of modern reasoning LLMs.

From a model-architecture perspective, existing methods address this bottleneck from two sides. **Output-side** methods predict more tokens per forward step: speculative decoding (Leviathan et al., 2023) predicts draft tokens with a small draft model, and verifies them with a large verifier model, which determines whether to accept draft tokens via rejection sampling; EAGLE (Li et al., 2024b) extends this idea to the hidden-feature level, and verifies a tree of candidates in parallel, yielding higher acceptance rates. Recent strong backbones incorporate multi-token prediction (MTP) heads directly into the model (Liu et al., 2024; Xiao et al., 2026; Team, 2026), which serve as co-trained drafters within the same speculative framework. **Input-side** methods reduce the effective sequence length with a compressor, merging multiple input tokens into one latent / continuous representation (Tan et al., 2026; Zhang et al., 2025; Tang et al., 2026), i.e., latent reasoning (Yu et al., 2026).

These two lines, however, have been pursued independently, and a unified design that exploits *both* sides is still missing. Moreover, output-side speedups remain bounded by the verifier’s forward-pass cost: every accepted token must still pass through the full backbone. To bridge these gaps, we start from two key observations.

Observation 1: a latent compressor and an MTP head are mirror images. A latent compressor folds two input token embeddings into one latent representation on the way in; an MTP head unfolds one hidden state into one additional output token on the way out. Combining them yields a symmetric **pair-in / pair-out** interface that simultaneously

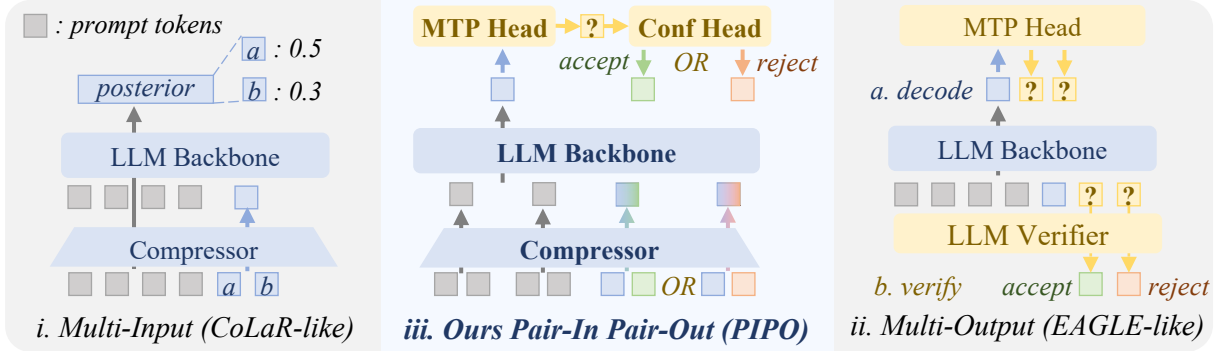


Figure 1: Comparison of input-side methods, output-side methods, and our proposed PIPO. PIPO treats a latent compressor and a multi-token prediction (MTP) head as mirror-image operations around the backbone, and trains a lightweight confidence head that replaces the verifier of speculative decoding (e.g., EAGLE).

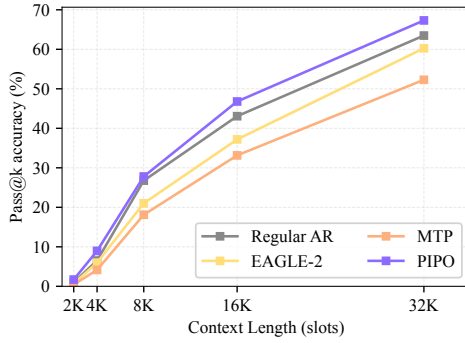


Figure 2: Overall pass@4 of PIPO and baseline methods on the four evaluation benchmarks, as the response context budget varies from 2K to 32K.

halves the effective input length and doubles the per-step output. We call this framework **Pair-In, Pair-Out (PIPO)** (Figure 1). Because MTP heads are already included in modern strong LLMs, PIPO can be built from mostly off-the-shelf components.

Observation 2: the on-policy distillation teacher is the speculative-decoding verifier. In **speculative decoding**, a strong LLM serves as a **verifier** that accepts a draft token x with probability $\min(p_v(x)/p_d(x), 1)$, where $p_v(x)$ is the verifier’s probability of x , and $p_d(x)$ is the draft model’s probability of x . In **on-policy distillation (OPD)** (Agarwal et al., 2024; Li et al., 2026), a strong model serves as a **teacher** to drive a reverse-KL distillation loss $\mathcal{D} = \text{KL}(p_s \parallel p_t)$, where p_t is the teacher’s probability and p_s is the student’s probability. These two roles play the same function: both judge whether the student / draft is consistent with a stronger teacher / verifier. PIPO exploits this by integrating a lightweight **confidence head**, trained with the rejection-sampling acceptance probability $\min(p_t/p_s, 1)$ as its label. The

supervision is essentially free: the same (p_t, p_s) distributions are already computed for OPD training. At inference time, the lightweight confidence head replaces the heavy verifier pass.

We evaluate PIPO on AIME 2025 (AMC, 2025), GPQA-Diamond (Rein et al., 2024), LiveCodeBench v6 (Jain et al., 2024), and LongBench v2 (Bai et al., 2025), using Qwen3.5-4B and Qwen3.5-9B backbones (Team, 2026), against regular decoding, MTP decoding without verification, and EAGLE-2 (Li et al., 2024a) speculative decoding. PIPO is the strongest pass@4 method on both backbones, improving the best baseline by +3.83 points on Qwen3.5-4B and +7.15 points on Qwen3.5-9B. As shown in Figure 2, this advantage *widens* as the context budget grows, because PIPO emits twice as many tokens per step, thus fits more complete chains of reasoning into the same budget. On the efficiency side, PIPO achieves up to a $2.64\times$ speedup in time-to-first-token (TTFT) and a $2.07\times$ speedup in time-per-output-token (TPOT) over regular decoding.

To conclude, our contributions are:

- We propose PIPO, which unifies input-side latent compression with output-side multi-token prediction for efficient LLM decoding.
- We observe that the OPD teacher and the speculative-decoding verifier play the same role, and exploit this to train a lightweight confidence head.
- Across four challenging benchmarks on Qwen3.5-4B and 9B, PIPO improves pass@4 by up to +7.15 points over regular decoding, while delivering up to $2.64\times$ TTFT and $2.07\times$ TPOT speedups.

2 Related Work

2.1 Output-side: Multi-token Decoding

Output-side accelerators aim to emit more than one token per forward step. *Speculative decoding* drafts candidate tokens with a smaller proposal model and verifies them by running a full forward pass through the target backbone (Leviathan et al., 2023). EA-GLE and its variants improve draft quality by drafting in feature space and verifying dynamic draft trees in a single parallel pass (Li et al., 2024b,a, 2025b). These methods preserve the target distribution exactly, but the verification pass through the large backbone is incurred at *every* decoding step, and at long context lengths this verifier cost can erase the gain from accepting multiple tokens at once.

A parallel line attaches additional prediction heads directly to the backbone. Medusa adds multiple decoding heads for parallel token prediction (Cai et al., 2024), and recent large models go further by natively including MTP modules during pretraining or post-training (Liu et al., 2024; Yang et al., 2025; Xiao et al., 2026), turning the MTP head into a standard, built-in component of modern LLMs. These heads reduce the number of decoding steps when their drafts are accepted, but without a verification or acceptance mechanism, unreliable drafts propagate into the generated sequence and hurt downstream accuracy.

PIPO builds on the strength of pretrained MTP heads and resolves the verifier bottleneck differently. Instead of an extra backbone forward pass per step, PIPO learns a lightweight *confidence head* whose supervision is recycled, for free, from the teacher distributions of on-policy distillation (see Section 3.4). The trained head replaces the per-step verifier entirely, turning a recurring inference cost into a one-time training signal.

2.2 Input-side: Latent Compression

Input-side methods reduce the number of effective input tokens by replacing or compressing token-level reasoning with continuous representations. Coconut feeds the backbone’s hidden states back to the model as continuous thoughts in place of decoded tokens (Hao et al., 2024). Soft Thinking and related analyses study soft or stochastic token representations in continuous concept space (Zhang et al., 2025; Wu et al., 2025; Tang et al., 2026). These methods change *what* the model attends to, but do not change *how many* tokens are emitted

per decoding step, and so leave the output-side bottleneck untouched.

CoLaR (Tan et al., 2026) is the closest work to PIPO: it compresses reasoning chains in latent space and directly predicts compressed latent embeddings. However, its latent prediction relies on a unimodal Gaussian assumption, which poorly fits the multi-modal distribution of multi-token continuations. PIPO keeps generation in the discrete vocabulary space, and uses latent variables only on the *input* side. This asymmetric design is what makes pair-in compression compatible with an off-the-shelf MTP head and a confidence-based acceptance rule. Additional related work on reasoning and reasoning-efficient LLMs is discussed in Appendix A.

3 Method

3.1 Overview and Notation

PIPO changes the decoding unit from a single token to a token *pair*. Let x^j denote the embedding at position j ; we use the same symbol for a *token* and its *embedding* when the meaning is clear, since the embedding layer is fixed.

As illustrated in Figure 3 (i), at pair step i PIPO receives a compressed latent input z^i that represents two consecutive tokens (x^{2i}, x^{2i+1}) , and predicts a backbone-token distribution p_b^{2i+2} together with a draft-token distribution p_d^{2i+3} . To gain efficiency without committing to unreliable drafts, PIPO also predicts a confidence score $c^i \in (0, 1)$. If $c^i \geq \tau_c$, the draft token is accepted and the next input pair contains two newly generated tokens. Otherwise, the draft token is rejected and replaced by a padding token before compression, so that PIPO keeps the same pair-level interface regardless of acceptance.

3.2 The Pair-In / Pair-Out Architecture

The PIPO architecture is built from two operations that are symmetric around the backbone: a compressor on the input side and an MTP head on the output side.

Pair-in compression. The compressor maps every two consecutive token embeddings into one latent input:

$$z^i = f_\theta([x^{2i}; x^{2i+1}]), \quad (1)$$

where $[\cdot; \cdot]$ denotes concatenation and f_θ is an MLP. We initialize f_θ so that $f_\theta([a; b]) \approx a + b$, which

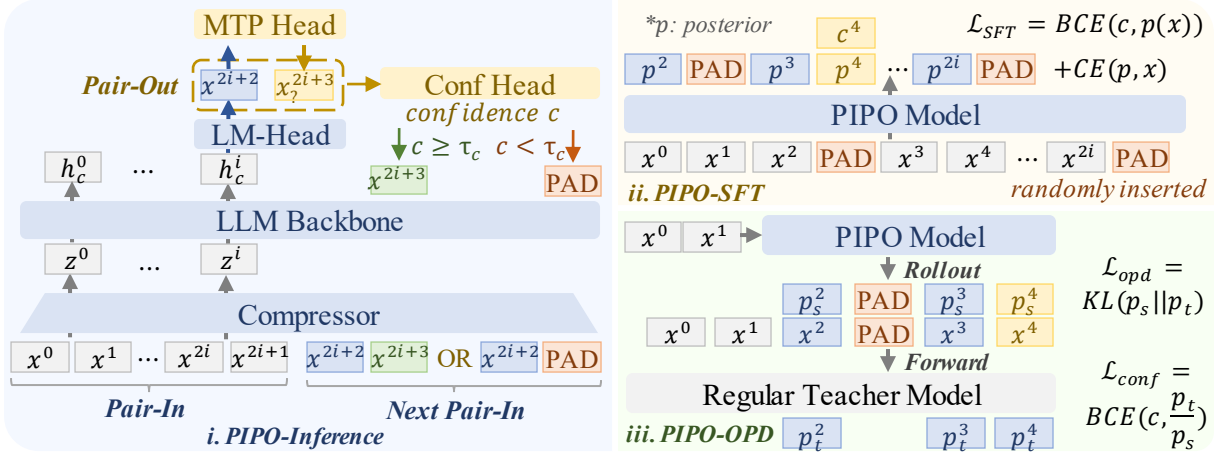


Figure 3: Illustration of PIPO’s architecture (i), SFT training process (ii), and OPD training process (iii).

keeps the backbone’s input distribution close to its pretraining distribution at the start of training (discussed in Section 4.4). Appendix E.1 probes the *trained* compressor and shows it preserves the *additive geometry* while learning a *non-symmetric, position-aware* projection.

Pair-out prediction. The LLM backbone produces a hidden state over the compressed prefix and a distribution over the next backbone token:

$$h_b^i = \text{Backbone}(z^{\leq i}), \quad (2)$$

$$p_b^{2i+2} = \text{LMHead}(h_b^i). \quad (3)$$

Then, the draft token is predicted by an MTP head, conditioned on the backbone hidden state and the embedding of the backbone token:

$$h_d^i = \text{MTPHead}(h_b^i, x^{2i+2}), \quad (4)$$

$$p_d^{2i+3} = \text{LMHead}(h_d^i). \quad (5)$$

Compression and MTP prediction are mirror-image operations: one folds two token embeddings into one latent input, and the other unfolds one hidden state into one additional output token. MTP heads are already included in modern strong LLMs, so the only new module that PIPO adds to the backbone is the small MLP compressor and confidence head (as discussed below). PIPO therefore obtains a pair-in / pair-out interface mostly from off-the-shelf components.

Confidence-guided draft acceptance. In speculative decoding, the draft token can be wrong and must be verified by a large verifier model, which is computationally expensive. PIPO instead integrates a lightweight confidence head that directly

estimates whether the draft token should be used:

$$c^i = g_\phi([h_b^i, h_d^i]), \quad (6)$$

where g_ϕ is a small MLP that produces a scalar in $(0, 1)$. If $c^i \geq \tau_c$, the next input pair is (x^{2i+2}, x^{2i+3}) ; otherwise it is $(x^{2i+2}, x^{\text{pad}})$, where x^{pad} is a fixed padding embedding. This rule provides a safe fallback to regular single-token decoding whenever the head is uncertain, while incurring only one extra MLP forward pass per pair instead of an entire backbone pass. Appendix E.2 verifies that the compressor effectively treats a padded pair as the surviving token alone, so the fallback does not pollute the backbone’s hidden state.

3.3 Supervised Fine-Tuning

SFT trains PIPO with a next-pair prediction objective. At each pair step i , the model predicts the ground-truth backbone token x^{2i+2} and draft token x^{2i+3} with a standard cross-entropy (CE) loss:

$$\mathcal{L}_{\text{tok}} = \text{CE}(p_b^{2i+2}, x^{2i+2}) + \text{CE}(p_d^{2i+3}, x^{2i+3}). \quad (7)$$

We additionally bootstrap the confidence head in this stage: its prediction c^i is aligned with the ground-truth draft token’s probability through a binary cross-entropy (BCE) loss,

$$\mathcal{L}_{\text{conf}}^{\text{SFT}} = \text{BCE}(c^i, p_d^{2i+3}(x^{2i+3})), \quad (8)$$

so that c^i already correlates with draft reliability before OPD provides a sharper supervision signal. The overall SFT objective is $\mathcal{L}_{\text{SFT}} = \mathcal{L}_{\text{tok}} + \lambda_{\text{conf}} \mathcal{L}_{\text{conf}}^{\text{SFT}}$. To match inference behavior, we randomly replace a fraction of the draft-position inputs with the padding embedding x^{pad} , exposing the model to rejected-draft cases during training.

3.4 On-Policy Distillation

SFT trains the pair-level interface, but never exposes PIPO to its own decoding distribution. We close this train–inference gap with on-policy distillation (OPD). Given a prompt, PIPO rolls out a response under its current policy, recording the accepted draft tokens, the padding tokens at rejected draft positions, and the student distributions p_s and confidence scores c at every step. We then remove the padding tokens, feed the resulting clean text to an uncompressed teacher, and obtain token-level teacher distributions p_t . The reverse-KL distillation loss aligns the student to the teacher,

$$\mathcal{L}_{\text{distill}} = \text{KL}(p_s \parallel p_t). \quad (9)$$

The teacher is the verifier. Speculative decoding accepts a draft token x with probability $\min(p_t(x)/p_s(x), 1)$, using a strong target model as the per-step verifier. In OPD, the same (p_t, p_s) pair is already computed at every position, i.e., exactly the quantities required by the verifier criterion. The OPD teacher and the speculative-decoding verifier therefore play the same role: both judge whether the student draft is consistent with a stronger reference distribution.

PIPO exploits this role-unification by training the confidence head with the rejection-sampling acceptance probability:

$$y_c = \min\left(\frac{p_t(x)}{p_s(x)}, 1\right), \quad (10)$$

$$\mathcal{L}_{\text{conf}}^{\text{OPD}} = \text{BCE}(c, y_c). \quad (11)$$

This supervision is essentially free: (p_t, p_s) are already computed for $\mathcal{L}_{\text{distill}}$, so the confidence head reuses them with no extra forward passes and no extra labels. At inference, the trained confidence head replaces the verifier pass entirely, turning a per-step inference cost into a one-time training signal. Overall, the OPD objective is

$$\mathcal{L}_{\text{OPD}} = \mathcal{L}_{\text{distill}} + \lambda_{\text{conf}} \mathcal{L}_{\text{conf}}^{\text{OPD}}. \quad (12)$$

4 Experiments

4.1 Experimental Setup

Training data. We train PIPO on DAPO-Math (Yu et al., 2025) (17.4k math questions) and Codeforces (Penedo et al., 2025) (16.1k coding questions), with a 90/10 SFT/OPD split. SFT trajectories come from sampling four responses per

question with Qwen3.5-9B (the teacher) and keeping all correct ones, yielding $\sim 90\text{k}$ trajectories of average length 24.4K tokens (capped at 64K; full statistics of SFT training data are in Appendix C). For OPD we additionally roll out four teacher responses per question to estimate difficulty and to drive the data filter studied in Section 4.4.

Training setting. PIPO is trained in two stages: 2 epochs of SFT with 25% random padding at draft positions (to expose the model to rejected-draft inputs), followed by 1 epoch of OPD on rollouts of the SFT student. Both stages use LoRA (Hu et al., 2022) adapters and AdamW with learning rate 1×10^{-4} , 5% warmup and cosine annealing; the confidence-loss weight is fixed to $\lambda_{\text{conf}} = 1.0$ throughout. Further details are elaborated in appendix C.

Evaluation data. We evaluate on four challenging benchmarks: (i) *AIME 2025* (AMC, 2025) consists of 30 math competition problems; (ii) *GPQA-Diamond* (Rein et al., 2024) consists of 198 graduate-level multiple-choice questions in physics, chemistry, and biology; (iii) *LiveCodeBench v6* (Jain et al., 2024) consists of 131 recent competitive-programming problems; and (iv) *LongBench v2* (Bai et al., 2025) (short subset due to the models’ context limit) consists of 178 long-context ($> 10\text{K}$ input) reasoning problems.

Evaluation setting. Following Qwen3.5 (Team, 2026), we use temperature = 1.0, top-p = 0.95, top-k = 20, and a repetition penalty of 1.5, with a 32K-slot response budget shared by all methods (the semantics of *slot* is discussed in Section 4.3). We sample four responses per question and report **avg@4** (mean accuracy) and **pass@4** (at least one of the four trials correct); further details are elaborated in Appendix D.

Baselines. All methods run on the same Qwen3.5-4B and 9B backbones (Team, 2026). We compare PIPO against (i) **Regular** autoregressive decoding; (ii) **MTP** (Team, 2026), which uses the pretrained MTP head to emit a draft token per step *without verification*, doubling per-step output at the risk of propagating unreliable drafts; and (iii) **EAGLE-2** (Li et al., 2024a), a strong speculative-decoding baseline that drafts with the MTP head and verifies the draft tree in one backbone forward pass. We report two PIPO variants: **PIPO-SFT** (SFT only) and **PIPO + OPD** (on-policy distillation post-trained on PIPO-SFT).

Table 1: Main results on AIME 2025, GPQA-Diamond, LiveCodeBench v6, and LongBench v2 (short). All methods share the same Qwen3.5-4B / 9B backbone and a 32K-slot response budget. **Bold** marks the best score in each column; underline marks the second best.

	AIME 2025		GPQA-Diamond		LiveCodeBench v6		LongBench v2		Overall	
	avg@4	pass@4	avg@4	pass@4	avg@4	pass@4	avg@4	pass@4	avg@4	pass@4
Qwen3.5-4B										
Regular	49.17	63.33	59.72	<u>72.73</u>	40.08	44.27	59.69	73.60	52.16	63.48
Eagle-2	40.83	60.00	53.66	66.16	32.06	43.51	<u>58.29</u>	71.35	46.21	60.26
MTP	34.17	43.33	52.27	69.19	14.89	27.48	52.67	69.10	38.50	52.28
PIPO-SFT	42.50	60.00	59.47	79.29	30.15	<u>48.85</u>	49.02	71.91	45.28	<u>65.01</u>
+ OPD	50.00	76.67	54.17	<u>72.73</u>	<u>32.06</u>	49.62	49.86	70.22	<u>46.52</u>	67.31
Qwen3.5-9B										
Regular	53.33	66.67	68.56	78.79	47.14	54.20	61.66	72.47	57.67	68.03
Eagle-2	49.17	63.33	62.63	73.23	39.31	45.80	<u>61.24</u>	71.35	53.09	63.43
MTP	40.00	50.00	55.30	71.21	18.32	31.30	52.81	71.91	41.61	56.10
PIPO-SFT	51.67	76.67	63.76	<u>81.31</u>	34.92	<u>54.20</u>	54.35	74.16	51.18	71.58
+ OPD	59.17	83.33	<u>67.17</u>	82.32	<u>46.56</u>	62.60	56.46	<u>72.47</u>	<u>57.34</u>	75.18

4.2 Main Results

To evaluate the effectiveness of PIPO, we compare it against three strong baselines on four challenging benchmarks. Table 1 reports the main results, from which we draw three key observations.

PIPO is the strongest pass@4 method on both backbones. Even without OPD, PIPO-SFT already surpasses every baseline on pass@4, improving over the best baseline (Regular) by +1.53 points on Qwen3.5-4B and +3.55 points on Qwen3.5-9B. Adding OPD widens the gain to +3.83 points on 4B and +7.15 points on 9B, with the best pass@4 in every per-task column except 4B LongBench. We attribute this to PIPO’s pair-in interface: under a fixed 32K-slot response budget, doubling the per-step output halves the effective length cost of a token, letting PIPO fit more complete reasoning chains into the same budget. The effect is most visible on AIME 2025, where PIPO + OPD lifts pass@4 by +13.34 and +16.66 points on 4B and 9B, respectively.

OPD recovers avg@4 while preserving the pass@4 gain. PIPO-SFT trades some avg@4 for higher pass@4, because doubling the per-step output adds uncertainty at draft positions. OPD closes this gap: +1.24 avg@4 / +2.30 pass@4 on 4B, and +6.16 avg@4 / +3.60 pass@4 on 9B. On the 9B backbone, PIPO + OPD matches Regular on avg@4 (57.34 vs. 57.67) while improving pass@4 by +7.15 points. Distillation from the teacher restores stability without sacrificing answer-space coverage. The SFT-to-OPD gain also grows with model size, suggesting that larger backbones bene-

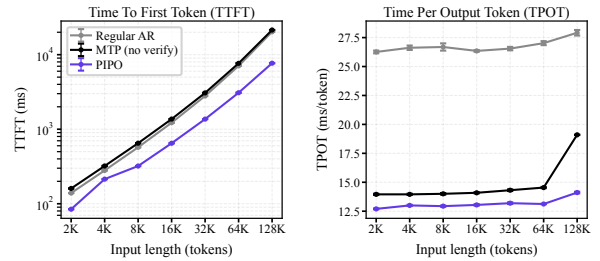


Figure 4: TTFT and TPOT on Qwen3.5-4B at different levels of input length, averaged over 16 trials.

fit more from OPD.

Existing accelerators still trade off accuracy.

MTP, which accepts every draft without verification, drops pass@4 by more than 11 points on both backbones, confirming that unverified drafts propagate errors. EAGLE-2, with its verifier in the loop, is closer to Regular but still 3–5 points worse on pass@4.¹ PIPO replaces this verifier with a single MLP per pair, gaining both higher pass@4 and the efficiency profile reported next.

4.3 Efficiency Analysis

We assess efficiency along two complementary axes: output slots per response (Table 2) and wall-clock latency on the HuggingFace backend at input lengths {2, 4, 8, 16, 32, 64, 128}K (Figure 4). For

¹EAGLE-2’s acceptance rule is distribution-preserving only under *exact* (greedy or temperature-only) speculative sampling (Leviathan et al., 2023); the truncation-based samplers we use (top-*p*, top-*k*, repetition penalty) fall outside this guarantee, so the draft no longer matches the verifier distribution, and practical acceptance becomes only approximately lossless. Small per-token drifts then accumulate over multi-thousand-token reasoning traces.

Table 2: Average output slots (# L) per response on Qwen3.5-4B/9B under a 32K-slot generation budget. Reg. = Regular, E-2 = EAGLE-2, PIPO-S/O = PIPO-SFT / PIPO + OPD.

# L	Reg.	E-2	MTP	PIPO-S	PIPO-O
4B	20,160	21,667	22,072	18,082	19,431
9B	19,140	20,584	21,857	17,494	18,590

wall-clock we report **TTFT** (time-to-first-token) and **TPOT** (time per output token, averaged over the first 16 generated tokens), each averaged over 16 trials.

Slot efficiency. A *slot* denotes one output unit at the method’s native granularity: one decoded token for Regular, EAGLE-2 and MTP (each emitted token occupies one slot); one *token-pair* for PIPO, which emits one backbone token and one draft token per step, which are then compressed back into one input latent for the next step. The tokens-per-slot ratio is therefore fixed at $1\times$ for baselines, but ranges between $1\times$ and $2\times$ for PIPO depending on the confidence-head acceptance rate. Under the shared 32K-slot generation budget, PIPO + OPD still uses *fewer slots* than baselines: $\sim 3\%$ fewer than Regular, $\sim 10\%$ fewer than EAGLE-2, and $\sim 13\%$ fewer than MTP on both backbones, with PIPO-SFT cutting further to -10% on 4B and -9% on 9B, while each slot still contributes *strictly more* reasoning content than a baseline slot. OPD trades a few additional slots for the higher pass@4 reported in Section 4.2.

TTFT. Regular decoding processes the full prompt during prefill, so TTFT scales with input length, from 0.139s at 2K to 20.3s at 128K. MTP adds the MTP-head pass and is marginally slower than Regular at every length. PIPO instead halves the effective prefill length by compressing every two input tokens into one, yielding a $1.65\times$ speedup over Regular at 2K that grows to $2.64\times$ at 128K (20.3s $\rightarrow$ 7.69s). The relative gain increases with input length, because prefill cost dominates more strongly in long-context regimes, exactly where reasoning workloads live.

TPOT. Per-token cost is dominated by a single backbone forward pass; since MTP and PIPO both emit two tokens per pass, their TPOT is roughly half of Regular at all lengths. Concretely, PIPO reaches a $2.07\times$ TPOT speedup at 2K (12.7 vs. 26.3) and $1.98\times$ at 128K (14.1 vs. 27.9), and is

Table 3: Ablations of the PIPO-SFT architecture and training data, evaluated by overall avg@4 / pass@4 on the four benchmarks of Table 1.

	Avg@4	Pass@4
PIPO-SFT (default)	45.28	65.01
- linear compressor	43.86	60.38
- shortest response only	42.88	59.87
- random compressor init.	38.74	57.73

consistently the fastest because its compressed prefix also shrinks the KV cache. Combined with the $2.64\times$ TTFT gain above, PIPO’s speedups concentrate in the long-context regime, which dominates inference cost for modern reasoning models.

4.4 Ablation Studies

To understand the contribution of each PIPO component, we perform ablations on the architecture and training data. Main results are shown in Table 3. All ablations are run on Qwen3.5-4B.

The compressor needs non-linearity. Replacing the MLP compressor with a single linear layer drops pass@4 by 4.63 points (65.01 $\rightarrow$ 60.38), confirming that a non-linear transformation is needed to fuse two heterogeneous token embeddings into one input latent that the backbone can consume.

Compressor initialization matters a lot. Initializing the MLP so that $f_\theta(a, b) \approx a + b$ keeps the backbone’s input distribution close to its pretraining distribution at the start of training; random initialization causes the largest drop in the table (-6.54 avg@4, -7.28 pass@4), confirming our claim in Section 3.2 that feeding out-of-distribution inputs to the backbone destabilizes early SFT.

SFT benefits from response diversity. Replacing our default “all-correct responses” data with the *shortest* correct response per question drops pass@4 by 5.14 points. Keeping multiple correct trajectories per question therefore supplies useful diversity for the next-pair objective, even though they share the same final answer.

OPD needs an accurate-but-not-trivial teacher. We keep only questions for which the teacher solves at least $\rho \in \{0\%, 25\%, 50\%, 75\%, 100\%\}$ of its four rollouts and re-train PIPO + OPD with each subset (Figure 5). Avg@4 grows monotonically with ρ , since a more correct teacher provides more reliable supervision for both the distillation loss and the confidence head. However, pass@4 peaks

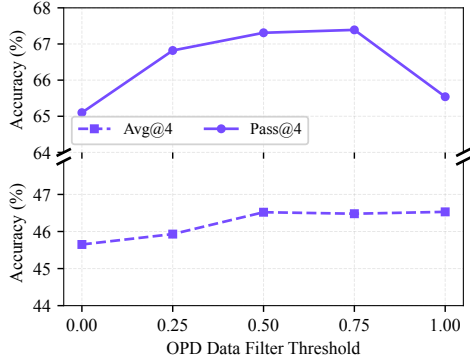


Figure 5: Effect of OPD data filtering by teacher correctness rate ρ (the minimum fraction of correct teacher rollouts required to keep a question in the OPD set).

at $\rho = 50\%$ and then drops: aggressive filtering removes the hardest questions, leaving the student under-exposed to the cases that matter most for answer-space coverage. We therefore use $\rho = 50\%$ in Table 1: stable enough to learn from, while preserving difficult questions the student must eventually solve.

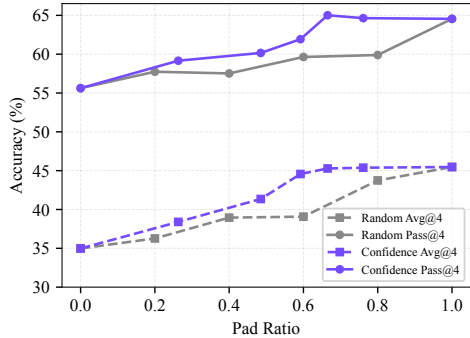


Figure 6: Overall avg@4 and pass@4 of PIPO-SFT on Qwen3.5-4B as a function of the *pad ratio*, i.e., the fraction of draft positions whose input is replaced by the padding embedding at the next pair step.

The confidence head is non-trivial and recovers a sweet spot. We sweep the acceptance threshold $\tau_c \in \{0, 0.5, 0.8, 0.9, 0.95, 0.98, 1.0\}$ (yielding pad ratios from 0 to 1) and compare this *Confidence* curve to a *Random* baseline that matches the same average pad ratios via an unconditional coin flip (Figure 6). Two observations stand out.

First, *Confidence dominates Random at every intermediate pad ratio*: e.g., at pad ~ 0.6 it reaches 61.93 pass@4 vs. Random’s 59.64, and the same gap holds throughout. This rules out the hypothesis that the head merely acts as an acceptance-rate knob; at the same acceptance budget it consistently rejects the drafts that matter and keeps the ones that

help.

Second, *Confidence peaks at an intermediate τ_c , not at the conservative extreme*: pass@4 maxes out at 65.01 for $\tau_c = 0.95$ (pad 0.665) and then drops to 64.55 as $\tau_c \rightarrow 1$ (single-token decoding). Notably, this peak *exceeds* the pad-1 baseline (64.55), meaning the accepted drafts under a well-tuned head do not merely preserve regular-decoding quality—they contribute additional, useful reasoning per step. Avg@4, in contrast, grows monotonically with pad ratio, reflecting the usual precision–coverage trade-off; we use $\tau_c = 0.95$ as the default in all main-table experiments.

5 Conclusion

We presented **PIPO**, a pair-in / pair-out framework for efficient LLM decoding that rests on two observations. First, a latent compressor and an MTP head are mirror-image operations on the two sides of the backbone; combining them yields a symmetric pair-level interface, which halves the effective input length and doubles the per-step output. Second, the on-policy distillation teacher plays the same role as the speculative-decoding verifier, so PIPO trains a lightweight confidence head with the teacher–student rejection-sampling ratio as a free label, amortizing the per-step verifier pass into a one-time training signal. On four reasoning, coding, and long-context benchmarks, PIPO improves pass@4 over regular decoding by up to +7.15 pp on Qwen3.5-9B, with up to $2.64\times$ TTFT and $2.07\times$ TPOT speedups.

6 Limitations

PIPO has several limitations, which we leave to future work. **First**, we study only the pair-in / pair-out setting. Larger compression factors may provide stronger speedups, but would be harder to model. **Second**, our experiments are limited to 4B–9B models due to compute constraints. However, PIPO exhibits higher performance gains on the larger 9B backbone, suggesting that it may be even more effective for larger models. **Third**, PIPO focuses on tasks with verifiable answers, and is not evaluated on open-ended generation tasks such as dialogue or creative writing. **Fourth**, PIPO is studied only for text-only models. Extending latent compression to multi-modal settings may require modality-specific compressors.

7 Ethics Considerations

PIPO is an inference-efficiency method for LLMs. It does not introduce new training data sources or new user-facing capabilities by itself. The main ethical impact is that faster decoding reduces inference cost and energy consumption, making reasoning models easier to deploy. At the same time, improved efficiency may also lower the cost of harmful uses of LLMs. The risks therefore largely follow those of the underlying base models and deployment settings. We recommend using PIPO with the same safety filters, monitoring, and access controls applied to the original backbones. Since confidence-based draft acceptance can affect generated content, practitioners should evaluate accuracy and safety on their target domains before deployment.

References

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. 2024. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, volume 2024, pages 21246–21263.
- AMC. 2025. https://artofproblemsolving.com/wiki/index.php/American_Invitational_Mathematics_Examination.
- Simon A Aytes, Jinheon Baek, and Sung Ju Hwang. 2025. Sketch-of-thought: Efficient llm reasoning with adaptive cognitive-inspired sketching. *arXiv preprint arXiv:2503.05179*.
- Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, and 1 others. 2025. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3639–3664.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*.
- Qian Cao, Xiting Wang, Yuzhuo Yuan, Yahui Liu, Fang Luo, and Ruihua Song. 2025. Evaluating text creativity across diverse domains: A dataset and large language model evaluator. *arXiv preprint arXiv:2505.19236*.
- Sicheng Feng, Gongfan Fang, Xinyin Ma, and Xinchao Wang. 2025. Efficient reasoning models: A survey. *arXiv preprint arXiv:2504.10903*.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shitong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. 2024. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, and 1 others. 2022. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3.
- Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, and 1 others. 2024. Openai o1 system card. *arXiv preprint arXiv:2412.16720*.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Live-codebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR.
- Jiaze Li, Hao Yin, Wenhui Tan, Jingyang Chen, Boshen Xu, Yuxun Qu, Yijing Chen, Jianzhong Ju, Zhenbo Luo, and Jian Luan. 2025a. Revisor: Beyond textual reflection, towards multimodal introspective reasoning in long-form video understanding. *arXiv preprint arXiv:2511.13026*.
- Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huan-ang Gao, Wenkai Yang, Zhiyuan Liu, and 1 others. 2026. Rethinking on-policy distillation of large language models: Phenomenology, mechanism, and recipe. *arXiv preprint arXiv:2604.13016*.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024a. Eagle-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pages 7421–7432.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024b. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning*.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025b. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Annual Conference on Neural Information Processing Systems*.

- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, and 1 others. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*.
- Guilherme Penedo, Anton Lozhkov, Hynek Krdlíček, Loubna Ben Allal, Edward Beeching, Agustín Piqueres Lajarín, Quentin Gallouédec, Nathan Habib, Lewis Tunstall, and Leandro von Werra. 2025. Codeforces. <https://huggingface.co/datasets/open-r1/codeforces>.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2024. [Gpqa: A graduate-level google-proof q&a benchmark](#). In *First Conference on Language Modeling*.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, and 1 others. 2024. Deepseek-math: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Hanjie Chen, and 1 others. 2025. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419*.
- Wenhui Tan, Jiaze Li, Jianzhong Ju, Zhenbo Luo, Ruihua Song, and Jian Luan. 2026. Think silently, think fast: Dynamic latent compression of llm reasoning chains. *Advances in Neural Information Processing Systems*, 38:4646–4668.
- Yao Tang, Li Dong, Yaru Hao, Qingxiu Dong, Furu Wei, and Jiatao Gu. 2026. Multiplex thinking: Reasoning via token-wise branch-and-merge. *arXiv preprint arXiv:2601.08808*.
- Qwen Team. 2026. [Qwen3.5: Accelerating productivity with native multimodal agents](#).
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.
- Junhong Wu, Jinliang Lu, Zixuan Ren, Gangqiang Hu, Zhi Wu, Dai Dai, and Hua Wu. 2025. Llms are single-threaded reasoners: Demystifying the working mechanism of soft thinking. *arXiv preprint arXiv:2508.03440*.
- Violet Xiang, Charlie Snell, Kanishk Gandhi, Alon Albalak, Anikait Singh, Chase Blagden, Duy Phung, Rafael Rafailov, Nathan Lile, Dakota Mahan, and 1 others. 2025. Towards system 2 reasoning in llms: Learning how to think with meta chain-of-thought. *arXiv preprint arXiv:2501.04682*.
- Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, Chiheng Lou, Fuli Luo, Gang Wang, and 1 others. 2026. Mimo-v2-flash technical report. *arXiv preprint arXiv:2601.02780*.
- Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. 2025. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, and 1 others. 2025. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*.
- Xinlei Yu, Zhangquan Chen, Yongbo He, Tianyu Fu, Cheng Yang, Chengming Xu, Yue Ma, Xiaobin Hu, Zhe Cao, Jie Xu, and 1 others. 2026. The latent space: Foundation, evolution, mechanism, ability, and outlook. *arXiv preprint arXiv:2604.02029*.
- Zhen Zhang, Xuehai He, Weixiang Yan, Ao Shen, Chenyang Zhao, Shuohang Wang, Yelong Shen, and Xin Eric Wang. 2025. Soft thinking: Unlocking the reasoning potential of llms in continuous concept space. *arXiv preprint arXiv:2505.15778*.
- Yuze Zhao, Jintao Huang, Jinghan Hu, Xingjun Wang, Yunlin Mao, Daoze Zhang, Zeyinzi Jiang, Zhikai Wu, Baole Ai, Ang Wang, Wenmeng Zhou, and Yingda Chen. 2024. [Swift: a scalable lightweight infrastructure for fine-tuning](#). *Preprint*, arXiv:2408.05517.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, and 1 others. 2025. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, and 1 others. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583.

A Extended Related Work

A.1 LLM Reasoning

Chain-of-Thought (CoT) prompting (Wei et al., 2022) encourages LLMs to produce explicit step-by-step traces before answering, and has been shown to substantially improve performance on complex tasks such as mathematics, code generation, long-form writing, and multimodal understanding (Cao et al., 2025; Li et al., 2025a). Recent “DeepThink” models (Jaech et al., 2024; Guo et al., 2025) go further by enclosing internal reasoning inside dedicated “`<think>/</think>`” tags before producing the final answer, a paradigm that has become the de-facto standard for the strongest open-weight reasoners. Reinforcement-learning-based post-training further amplifies the reasoning capability of these models: group-based methods such as GRPO (Shao et al., 2024) and its successors DAPO (Yu et al., 2025) and GSPO (Zheng et al., 2025) sample multiple candidate answers per prompt and re-weight them by correctness within each group.

A.2 On-policy Distillation

On-policy distillation (OPD) (Agarwal et al., 2024; Li et al., 2026) is a hybrid of supervised fine-tuning and distillation in which the student rolls out responses under its own current policy and is then aligned, position-by-position, to a frozen teacher’s distribution under a reverse-KL objective. Compared with standard SFT, OPD removes the train–inference distribution shift; compared with full-trajectory RL, it uses a dense per-token signal and avoids costly reward modeling. PIPO uses OPD both as a way to close the SFT–inference gap introduced by the pair-level interface and, more importantly, as the source of free supervision for its confidence head (Section 3.4).

A.3 Reasoning-efficient LLMs

Long chain-of-thought reasoning inflates inference cost (Wei et al., 2022; Xiang et al., 2025), and a growing literature studies how to shorten, compress, or adaptively terminate reasoning traces, ranging from prompting tricks (Xu et al., 2025; Aytes et al., 2025) to architectural modifications and length-aware training (Feng et al., 2025; Sui et al., 2025). PIPO is complementary to these efforts: it does not decide *how much* reasoning the model should perform, but reduces the per-token cost of *whatever* reasoning the model still produces.

The two directions can therefore be combined, e.g., a length-aware policy can be deployed on top of PIPO’s pair-level decoder to compound the savings.

B Architecture Details

B.1 Compressor Variants

The compressor maps a pair of consecutive token embeddings $(x^{2i}, x^{2i+1}) \in \mathbb{R}^{2H}$ into a single backbone-input latent $z^i \in \mathbb{R}^H$. We support two drop-in variants with identical input/output signatures (Table 4); the MLP variant is the default used in all main-table results. Both are initialized so that $f_\theta([a; b]) \approx a + b$ at step 0 (via zeroing the residual branch), which keeps the backbone’s input distribution close to its pretraining distribution at the start of training (the ablation in Section 4.4).

Table 4: Compressor variants. H is the backbone hidden size and $[\cdot; \cdot]$ denotes concatenation along the feature axis.

Variant	Formula
Linear	$W[x^{2i}; x^{2i+1}] + b$
MLP	$W_2 \text{SiLU}(W_1[x^{2i}; x^{2i+1}])$

B.2 MTP Head

The MTP head is a single full-attention decoder layer attached *after* the last backbone layer. Concretely, given the backbone hidden state h_b^i and the embedding of the just-decoded backbone token x^{2i+2} , we apply $h_d^i = \text{Layer}(W_{\text{fc}} [\text{RMSNorm}(h_b^i); \text{RMSNorm}(x^{2i+2})])$, where Layer is a Qwen3.5 decoder block sharing the backbone’s hyperparameters and $W_{\text{fc}} : \mathbb{R}^{2H} \rightarrow \mathbb{R}^H$ is a learnable projection. The same frozen LM head is then applied to h_d^i to produce the draft-token distribution p_d^{2i+3} . For the off-the-shelf MTP-equipped backbones used in this paper, the MTP layer ships pre-trained from Qwen3.5; PIPO only LoRA-adapts its attention and MLP projections and fully trains the small norm/projection modules (Section C.2).

B.3 Confidence Head

The confidence head g_ϕ takes the concatenated pair of backbone and MTP hidden states at pair step i and returns a scalar acceptance probability for the draft token:

$$c^i = \sigma(W_2 \text{SiLU}(W_1 \text{RMSNorm}([h_b^i; h_d^i]))) ,$$

with $W_1: \mathbb{R}^{2H} \rightarrow \mathbb{R}^H$ and $W_2: \mathbb{R}^H \rightarrow \mathbb{R}$ (no bias). On a 4B backbone this adds $\sim 6.6\text{M}$ parameters ($\approx 0.16\%$ of the model), and a single forward pass through the head is orders of magnitude cheaper than a full backbone verifier pass.

C Training Implementation Details

C.1 SFT Data

SFT trajectories are obtained by sampling four responses per question from the Qwen3.5-9B teacher on the union of DAPO-Math and Codeforces and keeping all correct trajectories, yielding 95,969 samples in total. Figure 7 shows the length distribution: mean 24.9K, median 21.6K, standard deviation 14.9K tokens, with a hard cap of 64K imposed at tokenization. The long right tail is the main reason why we cap evaluation at the 32K-slot budget shared with all baselines.

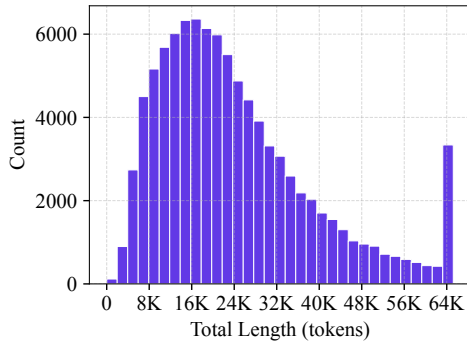


Figure 7: Token length distribution of the SFT corpus (95,969 Qwen3.5-9B trajectories on DAPO-Math and Codeforces).

C.2 SFT Training

We use ms-swift (Zhao et al., 2024) with LoRA (Hu et al., 2022) (rank 64, $\alpha = 128$, dropout 0.05) on the backbone projections $\{q, k, v, o, \text{gate}, \text{up}, \text{down}\}$ and, by name-suffix match, the same projections inside the MTP decoder layer. Beyond the LoRA adapters, we fully train the compressor MLP, the MTP projection W_{fc} , the MTP pre/post norms, and the confidence head; the LM head is tied to the input embedding and frozen. We train for 2 epochs on 8 H20-141G GPUs (per-device batch size 1, gradient accumulation 16) with DeepSpeed ZeRO-2, AdamW at learning rate 1×10^{-4} (5% linear warmup, cosine annealing), max sequence length 64K, and flash-attention 2. The MTP-loss weight λ_{mtp} and the confidence BCE weight λ_{conf} are both fixed at 1.

Random PAD injection. To expose the model to rejected-draft pairs at training time, we randomly inject the padding embedding at draft positions: at every step we sample $\rho \sim \text{Uniform}(0, \rho_{\text{max}})$ with $\rho_{\text{max}} = 0.25$, then independently mark a ρ fraction of eligible pairs (x^{2p}, x^{2p+1}) in the response region for splitting into $\{(x^{2p}, x^{\text{pad}}), (x^{2p+1}, x^{\text{pad}})\}$. Labels at PAD positions are masked from the cross-entropy and confidence losses. We additionally guarantee that PAD tokens always land at the odd position of a pair (so prompt/response boundaries align with pair boundaries) and that the total sequence length stays even, with the attention mask updated accordingly.

Confidence head target (SFT). During SFT, the teacher’s distribution at every label position is a one-hot δ_{y_t} over the ground-truth token, so the rejection-sampling acceptance probability collapses to

$$\begin{aligned} \sum_y \min(p_s(y), p_t(y)) &= \min(p_s(y_t), 1) \\ &= p_s(y_t), \end{aligned}$$

i.e., the student’s own probability on the gold draft token. The SFT-stage BCE target for the confidence head is therefore $p_d^{2i+3}(x^{2i+3})$, which is exactly the form used in Equation (7) of the main paper. Crucially, this target collapses to the same quantity as the OPD-stage rejection-sampling acceptance target in the deterministic-teacher limit, so the head transfers from SFT to OPD without a parameter reset.

C.3 OPD Training

OPD is a three-stage pipeline per micro-batch.

(1) Rollout: each question is rolled out by the SFT student under its own pair-level decoder via an SGLang colocate engine (Zheng et al., 2024), with the radix cache disabled to preserve PAD-augmentation determinism. **(2) Teacher forward (PAD compaction):** because the uncompressed Qwen3.5-9B teacher has never seen mid-sequence PAD tokens, we strip every PAD from the rolled-out trajectory, run the teacher on the clean sequence, and then re-map the teacher’s per-position log-probabilities back to the original (PAD-augmented) positions via a cumulative-sum lookup, where each PAD position inherits the teacher’s distribution conditioned on the immediately preceding non-PAD prefix, which is the correct conditional under our pair-in semantics. **(3) Student forward**

and loss: the student is re-forwarded on the same trajectory in compressed (pair-level) mode so that every trainable module (compressor, MTP head, LoRA on backbone and MTP, confidence head) receives gradient.

The default loss is a Monte-Carlo reverse-KL on the on-policy sampled tokens,

$$\mathcal{L}_{\text{distill}} = \mathbb{E}_{y \sim p_s} [\log p_s(y) - \log p_t(y)],$$

applied separately at even positions (backbone head) and odd positions (MTP head). We additionally support a top- k union mode (which restricts the divergence to the union of the student top- k , teacher top- k , and the sampled label, with $k=32$) and a full-vocab JSD mode; these are used only in early ablations. All KL/JSD computations are chunked along the token dimension with chunk size 2048, capping peak memory at $O(\text{chunk} \times V)$ instead of $O(T \times V)$.

Confidence head target (OPD). At the rolled-out draft position $y = x^{2i+3}$, the per-token rejection-sampling acceptance probability is

$$\begin{aligned} \alpha_i &= \min\left(\frac{p_t(y)}{p_s(y)}, 1\right) \\ &= \exp\left(-[\log p_s(y) - \log p_t(y)]_+\right), \end{aligned}$$

which is exactly the per-position quantity already computed by the sampled-KL loss above (modulo a clamp and an exp). The confidence head is therefore trained with a BCE loss against α_i as a one-time signal that recycles the same teacher/student forward passes used by the distillation loss; no extra teacher or student calls are introduced. We detach α_i from the autograd graph before BCE so that the head cannot leak gradient into the student’s logits via its own target.

OPD hyperparameters. We use 1 epoch, per-device batch size 1 with gradient accumulation 4, the same optimizer schedule as SFT, and LoRA rank 64 ($\alpha = 128$) on the same modules. Training runs on 8 H20-141G GPUs with $\text{tp} = 1$ and SGLang’s colocate rollout (model weights kept GPU-resident across rollout/training switches). We disable cross-sample padding inside the OPD chunk loop by running each sample through its own $B=1$ chunk and averaging.

D Evaluation Implementation Details

D.1 Inference Setup

All decoding runs use SGLang (Zheng et al., 2024) (with our LatentMTP extensions for PIPO) on $8 \times$ NVIDIA H20-141G GPUs, with data parallelism size 8, tensor parallelism size 1, and at most 64 concurrent requests per GPU. The response budget is set to 32K slots (Section 4.3) for every method, and the sampling parameters are fixed across methods as reported in Section 4.

D.2 Baseline Configurations

Regular. Plain autoregressive decoding through the Qwen3.5 SGLang backend.

MTP without verification. We run the Qwen3.5 model on the same SGLang backend with PIPO’s pair-out path enabled (so Qwen3.5’s MTP head produces a draft per backbone step), but without any verification; every draft token is therefore committed unconditionally.

EAGLE-2. We enable SGLang’s tree-based speculative decoding (NEXTN algorithm) with 3 speculative steps, top- $k = 1$ at every draft level, and 4 draft tokens per verifier call. The draft head is the off-the-shelf Qwen3.5 MTP head; the verifier is the full backbone.

PIPO. Both PIPO-SFT and PIPO + OPD are deployed with the confidence head active and $\tau_c = 0.95$, selected as the pass@4 sweet spot in the confidence-threshold sweep of Section 4.4.

D.3 Answer Evaluators

Models are instructed to put their final answer inside “\boxed{.}”. We extract the answer with a regular expression and, when several boxed expressions are present, take the last one. Mathematical answers (AIME 2025) are compared to the ground truth with the math-verify library;² multiple-choice answers (GPQA-Diamond, LongBench v2) use exact string match against the gold option label. LiveCodeBench v6 is execution-based: the extracted code is compiled and run against the benchmark’s hidden test suite, and the problem is counted as correct only if all tests pass.

E Additional Analyses

We complement the quantitative results of Section 4 with a probe of two of PIPO’s most distinctive com-

²<https://github.com/huggingface/Math-Verify>

ponents, the pair-in compressor (Appendix E.1) and the padding token (Appendix E.2). All analyses use the PIPO + OPD checkpoint on Qwen3.5-4B, pooled over 12 prompts (4 each from AIME 2025, GPQA-Diamond, and LiveCodeBench v6), for a total of 1,372 token pairs.

E.1 What does the compressor learn?

The compressor $f_\theta: \mathbb{R}^{2H} \rightarrow \mathbb{R}^H$ is initialized so that $f_\theta([a; b]) \approx a + b$ (Appendix B.1); a natural question is what it deviates to after training. We answer this with three complementary probes: (i) per-input Jacobian-norm sensitivity, (ii) swap-test cosine, and (iii) compressor-vs-sum alignment.

The compressor is position-unbiased. For each pair we measure the share of the output sensitivity attributable to the first input, $\|\partial f_\theta / \partial x^{2i}\| / (\|\partial f_\theta / \partial x^{2i}\| + \|\partial f_\theta / \partial x^{2i+1}\|)$. Figure 8 shows that this ratio concentrates tightly around 0.49, statistically indistinguishable from the symmetric value 0.5. The compressor therefore systematically over-weights neither the leading nor the trailing token in a pair, mirroring its symmetric additive initialization rather than collapsing onto one position.

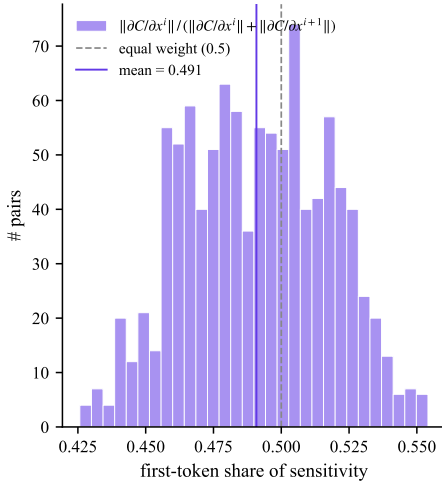


Figure 8: Per-pair sensitivity share of the first input, $\|\partial f_\theta / \partial x^{2i}\| / (\|\partial f_\theta / \partial x^{2i}\| + \|\partial f_\theta / \partial x^{2i+1}\|)$. The distribution is tightly concentrated near 0.5 (mean 0.491): the compressor weights both pair positions roughly equally.

The compressor is position-aware. If the compressor were a symmetric function (e.g., its sum-initialized starting point), swapping the two inputs would leave the output unchanged. We measure $\cos(f_\theta([x^{2i}; x^{2i+1}]), f_\theta([x^{2i+1}; x^{2i}]))$ for ev-

ery pair and compare to the input baseline $\cos(x^{2i}, x^{2i+1})$ that describes how similar the two raw embeddings already are. Figure 9 shows the swap cosine concentrates at 0.68, well below the order-invariant value 1.0 and far above the input baseline of 0.16. The compressor therefore moves substantially away from the symmetric solution during training and learns to encode *which* slot each token occupies in addition to *which* two tokens it received.

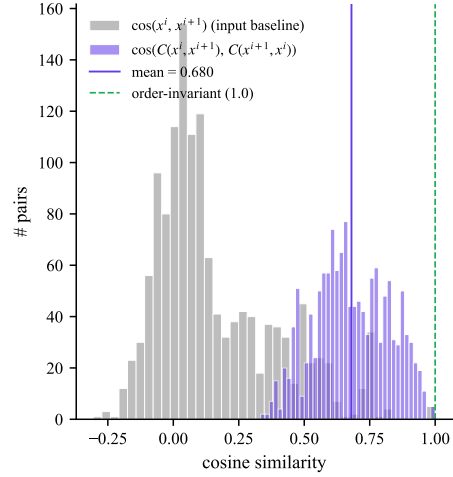


Figure 9: Swap-test cosine $\cos(f_\theta([a; b]), f_\theta([b; a]))$ (purple, mean 0.68) against the input baseline $\cos(a, b)$ (gray, mean 0.16). A fully order-invariant compressor would sit at 1.0 (green dashed); the trained compressor sits much lower, showing that it encodes slot identity.

The compressor learns beyond a sum. The previous two probes still leave open whether the trained compressor is essentially an additive map with a learned positional twist. Figure 10 shows that the compressor output is closer to the additive baseline $x^{2i} + x^{2i+1}$ (mean cosine 0.65) than to either constituent alone (0.49 and 0.51), but the gap from 1.0 is large and the output magnitude is sharply attenuated ($\|f_\theta(\cdot)\| / \|x^{2i} + x^{2i+1}\| \approx 0.50$ on average). Taken together with the swap-test, this confirms that f_θ has moved well past its $a + b$ initialization—it preserves the additive geometry that keeps the backbone in distribution while learning a sharper, position-aware projection that brings the pair embedding into a regime the backbone can decode at the pair granularity.

E.2 What is the role of the padding token?

The padding embedding plays a structural role at both training time (random PAD injection, Appendix C.2) and inference time (the next input pair

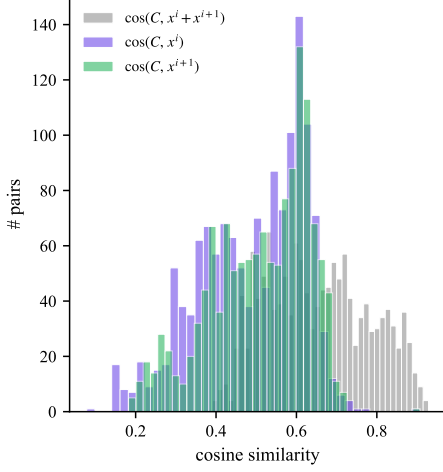


Figure 10: Cosine similarity between the compressor output and three references: the naive sum $x^{2i} + x^{2i+1}$ (gray), the first input alone (purple), and the second input alone (green). The output aligns most with the sum but stays clearly below 1.0.

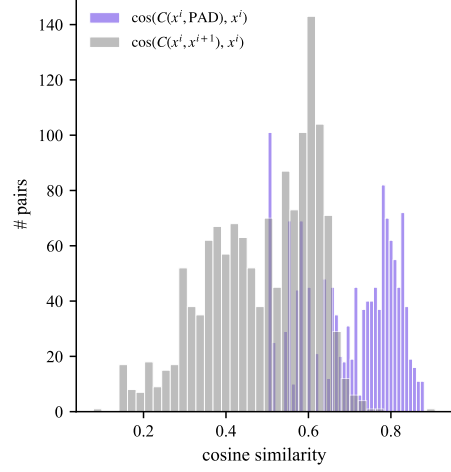


Figure 11: Effect of replacing the second pair token with PAD. Purple: $\cos(f_\theta([x^{2i}; \text{PAD}]), x^{2i})$, mean 0.70. Gray: $\cos(f_\theta([x^{2i}; x^{2i+1}]), x^{2i})$, mean 0.49. A PAD-injected pair aligns much more strongly with the surviving token, i.e., PAD is effectively ignored.

after a rejected draft is $(x^{2i+2}, x^{\text{pad}})$, Section 3.2). For PIPO to keep the same pair-level interface across both regimes, the compressor should treat a PAD-padded pair as “the surviving token alone” rather than as an out-of-distribution input. We verify this from two angles.

PAD is effectively ignored. Figure 11 replaces the second token of every pair by the PAD embedding and compares the compressor output to the surviving first token. The PAD-injected output aligns much more strongly with the surviving token ($\cos = 0.70$) than the unmodified output does ($\cos = 0.49$): when one slot is PAD, the compressor effectively delegates the pair latent to the non-PAD slot. The same behavior holds in reverse—putting PAD in the first slot raises $\cos(f_\theta([\text{PAD}; b]), b)$ to 0.60, well above the additive baseline.

All-PAD pairs collapse to a near-zero signal. Figure 12 plots the magnitude of the compressor output along a representative trajectory and compares it to several controls. The pure-PAD output $\|f_\theta([\text{PAD}; \text{PAD}])\| = 0.39$ (green dashed) is an order of magnitude below every other curve, including the PAD-on-one-side output (orange). This makes the all-PAD pair behave almost like a no-op KV-cache entry for the backbone: it has the right shape but carries vanishing signal, which is exactly the property needed for PIPO to fall back to single-token decoding under aggressive rejection

(Section 4.4) without polluting the hidden state of subsequent slots.

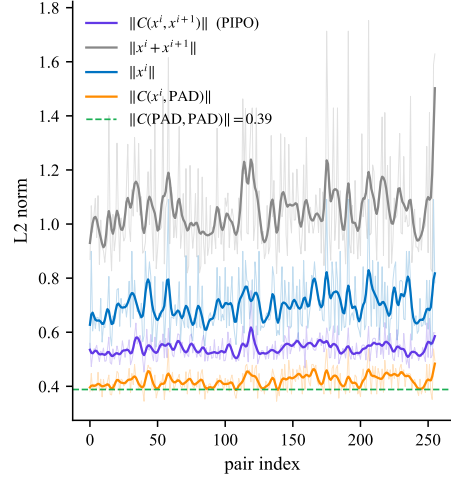


Figure 12: Compressor output norm along the longest probe trajectory, with all-PAD norm overlaid as a green dashed line. The all-PAD output is an order of magnitude smaller than every alternative, making it act as a near-zero signal to the backbone.